

This is a copy of a shared ChatGPT chat. It won't be added to memory.

Report conversation

Today 5:00 PM

📁 Uploaded a file

This is the current architecture layer of Productlens2.0.
There are a lot of issues currently in the implementation,
everytime I start a run it always fails at one or the other
stage. I do not understand what I am doing wrong

Yes. After reading the architecture, I think I can explain why you are experiencing the
"every run fails at one stage or another" problem.

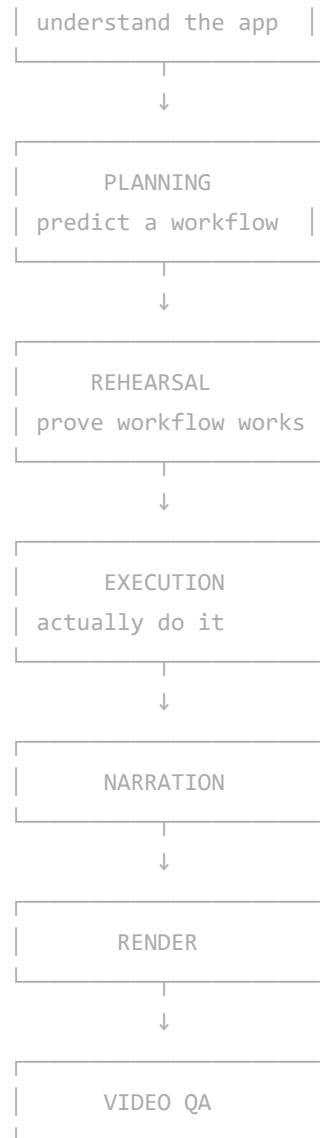
The important thing is: **I don't think your main problem is that ProductLens 2.0 has the wrong architecture.** The architecture is actually quite sophisticated. The problem is that you have built a very large number of reliability gates around an interaction engine that is **still not reliable enough at its core.**

Your README itself essentially admits this: the interaction layer is fail-closed, but arbitrary-app behavior still requires live rehearsal, and complex mutations remain the main unresolved capability. Pasted markdown

The fundamental problem

Right now ProductLens looks like this:

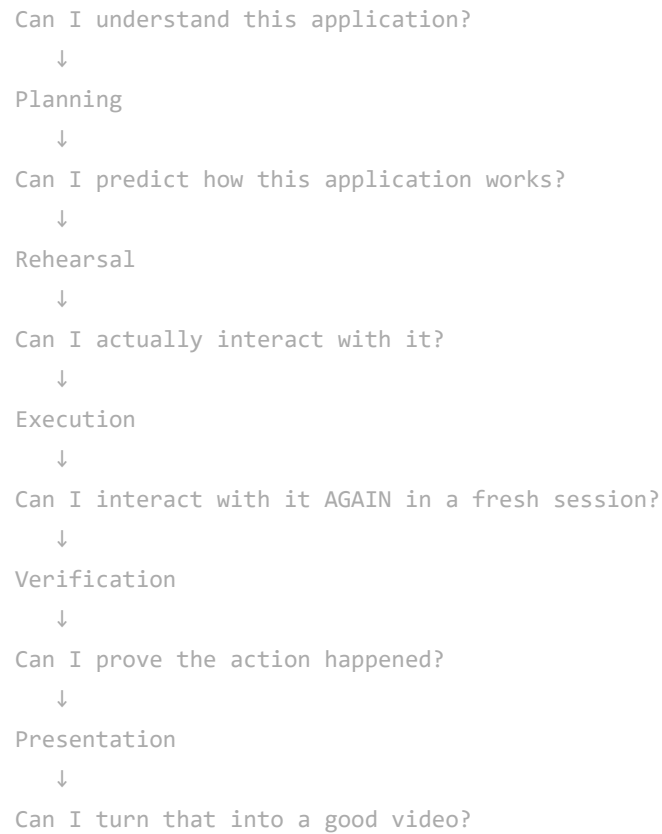
DISCOVERY



That looks excellent architecturally.

But underneath it, you have:

Discovery
↓



The interaction uncertainty propagates through the entire pipeline.

If interaction is 70–80% reliable, the architecture around it doesn't save you. Instead, the architecture gives you increasingly sophisticated ways of discovering that the interaction wasn't reliable.

1. You have too many things that can fail

This is probably the biggest practical issue.

You currently have:

ChatGPT ▾

- discovery
- authentication
- DOM extraction
- accessibility extraction
- Stagehand
- product knowledge
- feature graph
- candidate flows
- planning
- rehearsal
- certified workflow graph
- state graph
- execution
- target grounding
- action dispatch
- postcondition verification
- adaptive replanning
- trace generation
- presentation planning
- editorial enrichment
- narration
- caption validation
- rendering
- synchronization QA
- visual QA
- multimodal QA
- delivery QA
- automatic repair

 Share

Log in

Sign up for free

Your own architecture lists all these layers explicitly. Pasted markdown

Individually, they make sense.

Collectively, **they create an enormous failure surface.**

For example:

```
Discovery succeeds
↓
Planning succeeds
↓
Rehearsal succeeds
↓
Execution fails
↓
repair
↓
Execution succeeds
↓
Narration fails
↓
repair
↓
Narration succeeds
↓
Render fails
↓
repair
↓
Video QA fails
```

And you end up thinking:

"Why can't I get one run through?"

Because one successful run currently requires many independent contracts to hold simultaneously.

2. The architecture is ahead of the product

This is the part I would pay the most attention to.

You have built something resembling a **production-grade distributed workflow system** before solving the fundamental research/engineering problem completely.

For example, you have:

- ✓ durable stage jobs
- ✓ retry lineage
- ✓ repair children
- ✓ leases
- ✓ heartbeats
- ✓ artifact manifests
- ✓ delivery reports
- ✓ multimodal QA
- ✓ synchronization QA
- ✓ editorial preflight

That's good engineering *after* the underlying capability is stable.

But ProductLens's hardest problem is still:

- ✓ Given an unfamiliar application in its current state, determine what the user means and reliably manipulate the correct UI element.

Your own document identifies the interaction layer as the "primary reliability boundary."

Pasted markdown

And later it says broad arbitrary-app action selection still requires further live behavioral rehearsal.

Pasted markdown

That is the product.

Everything else is supporting infrastructure.

3. Your closed-loop executor is conceptually correct

I actually like this:

```
observe
↓
propose semantic candidate
↓
resolve unique visible target
↓
safety/preconditions
↓
dispatch
↓
capture resulting state
↓
verify independent witness
↓
re-observe
↓
next action
```

That's exactly the direction I would want. Pasted markdown

But there's a subtle problem.

Your executor assumes that the observation → target resolution → action mechanism chain is sufficiently good.

Imagine:

"Click the Add Lead button."

The system needs to determine:

```
What does "Add Lead" refer to?  
↓  
Which visible object is it?  
↓  
Is it actually clickable?  
↓  
Is it covered by another element?  
↓  
Does clicking it open a modal?  
↓  
Did the modal actually open?  
↓  
Is the modal the expected one?
```

Now consider:

"Create a lead."

That becomes much harder:

Find Leads
↓
Open creation interface
↓
Understand fields
↓
Determine required fields
↓
Fill them
↓
Discover dependent fields
↓
Select values
↓
Submit
↓
Determine whether submission happened
↓
Find resulting entity
↓
Verify entity

Your architecture has mechanisms for almost every one of those steps.

But mechanisms aren't capability.

4. Rehearsal is amplifying the problem

This is another major point.

You intentionally do:



For a create flow, rehearsal must prove the workflow before production. Pasted markdown

That's a good reliability principle.

But it introduces a very difficult requirement:

The agent must successfully solve the same unfamiliar application twice.

Suppose your interaction engine has:

`90% chance of successfully completing workflow`

Then two independent runs are roughly:

$$0.9 \times 0.9 = 81\%$$

At 80%:

$$0.8 \times 0.8 = 64\%$$

At 70%:

$$0.7 \times 0.7 = 49\%$$

And then add discovery, planning, authentication, verification, rendering, etc.

You can see why your end-to-end experience feels awful.

5. You're also testing two different things at once

This is subtle.

You have:

Capability reliability

Can ProductLens operate this application?

and:

Delivery reliability

Can ProductLens produce a valid demo video?

These are different.

Right now a run failing at:

```
REHEARSAL_OUTCOME_UNVERIFIED
```

and a run failing at:

CAPTION_TRACE_MISMATCH

both feel like:

"ProductLens failed."

But they are completely different engineering problems.

Your architecture correctly separates them conceptually, but **your development workflow should separate them much more aggressively.**

6. I would stop running the entire pipeline while debugging

This is probably the most important recommendation I can give you.

Do not use the full ProductLens pipeline as your development test.

Instead, build a hierarchy.

Level 1 — Browser capability

Forget ProductLens.

Give the system:

URL
objective

and ask:

Can the agent complete the interaction?

Output:

JSON

```
{  
  "success": true,  
  "actions": [...],  
  "observations": [...],  
  "outcome": {...}  
}
```

No video.

No narration.

No Remotion.

No captions.

No delivery QA.

No editorial layer.

Level 2 — Workflow certification

Now:

Discovery



Interaction agent

↓
Verified workflow

Ask:

Can I reliably reproduce this workflow?

Run it 10 times.

You want:

10/10

or at minimum understand precisely where the failures occur.

Level 3 — Production execution

Once the interaction works:

certified workflow
↓
fresh browser
↓
production execution
↓
trace + recording

Now test whether the exact same workflow can be filmed.

Level 4 — Presentation

Only then:

```
trace
↓
storyboard
↓
captions
↓
render
```

Level 5 — Delivery QA

Finally:

```
video
↓
QA
↓
delivery
```

This gives you a dramatically better debugging signal.

7. Your current architecture has an important circular dependency

Look at this:

```
Discovery
↓
```



But your interaction engine is supposed to work on unfamiliar applications.

So you're effectively saying:

Discovery tells me how the application works.

Then:

Planning predicts how to interact.

Then:

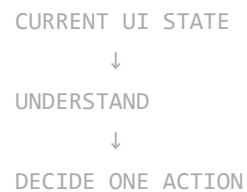
Rehearsal validates the prediction.

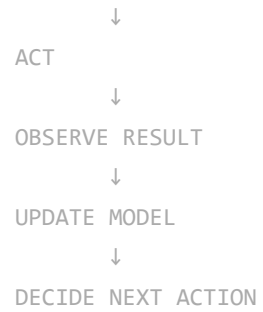
Then:

Production assumes the validated prediction is still valid.

This is fundamentally **model-based automation**.

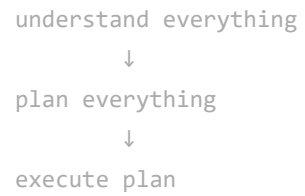
But what you really want for ProductLens is closer to:



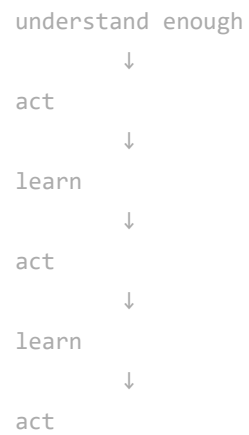


The model should continuously evolve during execution.

Not:



but:



Your current engine actually moves in this direction with re-observation and adaptive replanning, but I think **this should become the center of ProductLens rather than a feature inside the larger pipeline.**

8. I think your "planner" is doing too much

You currently have:

```
Objective
↓
candidate flows
↓
scoring
↓
workflow
↓
state graph
↓
certified workflow
↓
production plan
```

That's a lot of abstraction between:

"I want to create a lead"

and:

"click this thing."

This creates opportunities for stale or incorrect assumptions.

I'd simplify the mental model to:



The planner should not necessarily decide the entire execution sequence upfront.

It should decide:

What should I accomplish next given what I can currently observe?

9. Your knowledge layer can become dangerous

You have:

- ProductKnowledge
- PageKnowledge
- FeatureKnowledge
- relevance graph
- behavioral model
- candidate flows
- validated state graph

That's powerful.

But every additional persisted model creates another opportunity for:

```
old understanding
  ↓
new application state
  ↓
wrong assumption
  ↓
wrong action
```

You do have freshness/fingerprinting/re-grounding mechanisms. Pasted markdown

That's good.

But I'd make a very strong distinction:

Knowledge

"What I previously learned."

versus

Truth

"What I can observe right now."

For execution:

current browser state must always beat stored knowledge.

10. Your biggest debugging improvement: create a "failure ladder"

Instead of:

```
Run failed
```

you need something like:

```
TASK
↓
DISCOVERY
  ✓ URL loaded
  ✓ authenticated
  ✓ target feature found
↓
UNDERSTANDING
  ✓ target control found
  ✓ target semantics understood
↓
ACTION
  ✓ locator resolved
  ✓ action dispatched
↓
STATE CHANGE
  ✓ UI changed
↓
OUTCOME
  ✗ expected entity not found
↓
CLASSIFICATION

ROOT CAUSE:
Outcome verification
```

NOT:
Execution failure

This is enormously important.

Your system already produces things like `action-attempts.json` , `verification-results.json` , state snapshots and trace data. Pasted markdown

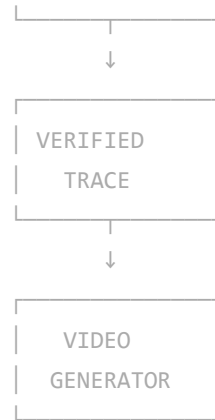
The next step isn't necessarily adding more QA.

It's making the **failure diagnosis** extremely precise.

11. I would simplify ProductLens 2.0 temporarily

If I were working on your codebase with you, I would temporarily reduce the system to:





Everything else can remain in the repository, but **don't let it participate until the core works.**

12. And this is where I think your earlier "AI agent" idea becomes interesting

You were recently considering:

If ProductLens isn't solved, perhaps the YC application should be about an AI agent that solves the interaction problem.

After reading this architecture, I understand much better why you were thinking that.

Because **the real technology you've been struggling to build isn't the video generator.**

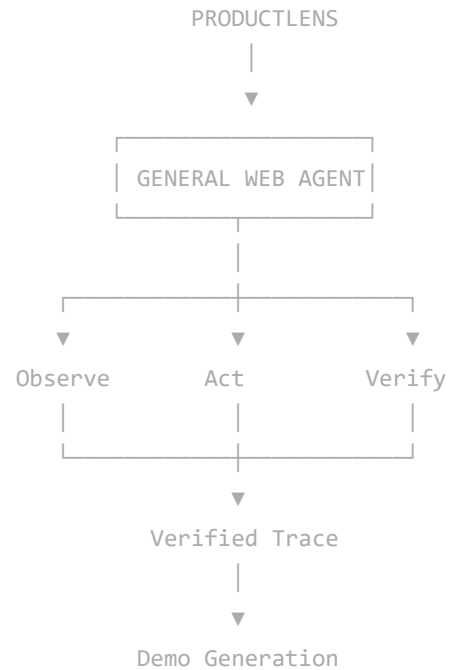
It's:

An agent that can understand and operate unfamiliar software reliably.

ProductLens is essentially a very demanding application of that technology.

The architecture currently makes this distinction somewhat blurry.

I'd invert it conceptually:



Then your core asset isn't:

"a complicated pipeline that makes demos."

It's:

"A reliable agent that can operate unfamiliar software and produce an evidence-backed execution trace."

The demo generator becomes the first killer application.

13. What I would NOT do right now

I would **not** do another huge refactor.

Your architecture assessment itself says line-count reduction isn't the success criterion and identifies the generation coordinator, discovery, editorial, execution adapter/engine, repository and rendering as complexity hotspots. Pasted markdown

But I don't think now is the time to split another 20 files.

You risk:

```
bad interaction architecture
      +
more abstractions
      =
harder debugging
```

Instead:

Freeze the architecture.

Then identify **one exact failing workflow**.

For example:

```
SmartSevak
→ Leads
→ Add Lead
→ fill fields
→ submit
→ verify created lead
```

And instrument it brutally.

14. Then measure every action

You should eventually be able to see something like:

ACTION 1

Intent:

Open Leads

Observation:

3 buttons

2 links

1 navigation item

Candidate:

"Leads"

Resolution:

role=link, name="Leads"

Dispatch:

click

After-state:

URL changed /leads

Verification:

PASS

Then:

ACTION 2

Intent:

Create a lead

Observation:

"Add Lead" button

Resolution:

button[name="Add Lead"]

Dispatch:

click

After-state:

dialog appeared

Verification:

PASS

Then:

ACTION 3

Intent:

Fill phone number

Observation:

3 textboxes

Candidate:

textbox[label="Phone"]

Resolution:

UNIQUE

Dispatch:

fill

```
After-state:  
value=...
```

```
Verification:  
PASS
```

If it fails, you know **exactly where and why**.

That's far more valuable right now than another layer of QA.

15. The most important architectural shift

I would make this principle explicit:

ProductLens does not need to understand an entire application before it starts acting.

It needs to understand **enough of the current state to choose the next safe action**.

Then:

```
observe  
→ reason  
→ act  
→ verify  
→ update understanding  
→ repeat
```

That is the loop I would spend the majority of your engineering effort on.

Your existing architecture already contains almost all the pieces necessary for this. The problem is that they are currently embedded inside a much larger pipeline.

My diagnosis in one sentence

You aren't failing because ProductLens 2.0 is insufficiently engineered; you're failing because you've engineered the reliability infrastructure around a core interaction capability that is still probabilistic, and then you're asking that capability to succeed across multiple independent browser sessions and dozens of downstream contracts.

So I would stop trying to make the whole pipeline reliable first.

Make this reliable first:

```
URL + objective
  ↓
observe
  ↓
understand
  ↓
choose action
  ↓
execute
  ↓
verify
  ↓
repeat
  ↓
VERIFIED TRACE
```

If you can get that loop genuinely reliable on unfamiliar applications, most of the rest of ProductLens becomes an engineering problem rather than the existential problem you're fighting right now.

And I think this also gives you a much cleaner way to decide the YC direction later:
ProductLens can be the product/application demonstrating the general agent capability, rather than the enormous pipeline itself being the core invention.



Sources

Yes — it covers the core discussion, but I would make one important distinction: it is primarily an interaction-engine fix, not a complete “fix every layer” implementation plan.

Your current architecture has two different problems:

1. The core interaction engine is unreliable.
2. The surrounding pipeline has too many coupled failure points.

The brief I gave Cursor addresses #1 **very strongly** and gives it the direction for #2, but I would add an explicit **whole-pipeline stabilization section**.

Your architecture already separates Discovery → Planning → Execution → Narration → Render → VIDEO_QA, and explicitly says those stages should not silently replace one another.

Pasted markdown

What the current brief covers

Layer	Covered?	What it addresses
Discovery	✓	Current-state observation, knowledge vs truth
Planning	✓	Don't over-plan; planning should support execution
Rehearsal	✓	Reliable interaction before production
Interaction	✓ ✓ Core focus	Observe → reason → act → verify
Hidden UI	✓ ✓	Progressive UI discovery
Execution	✓	Fresh-state execution + verification
Outcome verification	✓ ✓	Dispatch ≠ success
Complex apps	✓	Later phase for n8n/Excalidraw
Narration	⚠ Partial	Mostly treated as downstream
Rendering	⚠ Partial	Mostly treated as downstream
VIDEO_QA	⚠ Partial	Preserved, but not deeply redesigned
Repair system	⚠ Partial	Failure ownership discussed
Persistence/artifacts	⚠ Partial	Existing contracts should be preserved
Full pipeline reliability	⚠ Needs explicit plan	This is the missing piece

And I would not ask Cursor to “fix all layers simultaneously.”

That's exactly how you could end up back where you are now.

What I think you should tell Cursor

Add this section to the end of the previous prompt:

ProductLens 2.0 — Whole Pipeline Stabilization Addendum

Whole Pipeline Stabilization Addendum

The interaction engine is the highest-priority problem, but the implementation must also stabilize the entire ProductLens pipeline.

Do not redesign all layers simultaneously.

The goal is to make each layer have a clear contract, a clear owner, deterministic inputs/outputs, and precise failure reporting.

1. Pipeline principle

The pipeline must behave as:

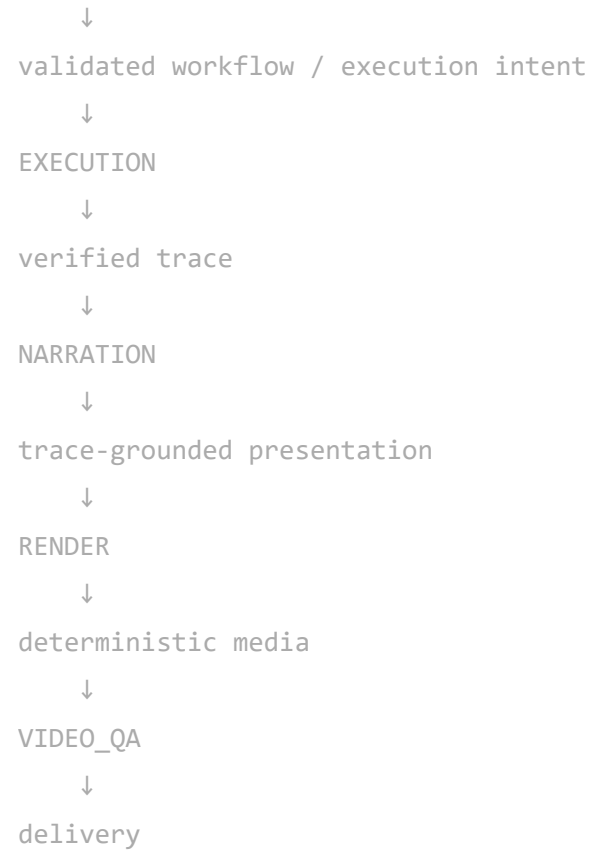
DISCOVERY

↓

validated evidence

↓

PLANNING



A downstream stage must never compensate for an upstream failure.

Examples:

execution failed

→ narration must not pretend it succeeded

verification failed

→ presentation must not treat the action as completed

render failed

→ QA must not reinterpret the source trace as a successful delivery

missing evidence

→ planner must not invent it

2. Every stage needs an explicit contract

For every durable stage, identify:

Inputs

Outputs

Required artifacts

Preconditions

Success criteria

Failure categories

Retry boundary

Owner

The six durable stages remain:

DISCOVERY

PLANNING

EXECUTION

NARRATION

RENDER

VIDEO_QA

Do not introduce additional durable stages unless there is a strong architectural reason.

Internal sub-steps are allowed.

3. Discovery contract

Discovery should answer:

"What can I reliably observe about this application right now?"

Discovery should produce evidence, not assumptions.

Required outputs include:

- product context
- page knowledge
- relevant controls
- forms
- navigation
- observed interaction patterns
- authentication state
- blockers
- candidate workflows
- behavioral evidence

Discovery should NOT attempt to solve the entire workflow.

It should provide enough grounded evidence for planning and interaction.

Discovery must distinguish:

OBSERVED

INFERRED

UNKNOWN

BLOCKED

Never turn inferred or unknown behavior into observed behavior.

4. Planning contract

Planning should answer:

"Given the evidence, what workflow appears capable of satisfying the objective?"

Planning should NOT fabricate:

- routes
- buttons
- fields
- canvas coordinates
- menu structures
- application-specific behavior

Every important workflow step should have:

intent

target evidence

preconditions

postconditions

importance

completion criteria

Planning can propose a workflow.

Execution remains responsible for determining whether the current browser state actually supports the proposed action.

Planning must tolerate the fact that some UI controls are initially hidden.

Do not require the planner to know every intermediate hidden control upfront.

5. Rehearsal contract

Rehearsal exists to answer:

"Can this workflow actually be completed safely in a fresh session?"

Rehearsal should use the same interaction principles as production:

observe

→ resolve

→ act

→ verify

→ re-observe

Rehearsal must produce a strong outcome witness.

A workflow is not certified merely because:

- a click succeeded
- a URL changed
- a modal appeared
- Stagehand suggested an action

It must have an independent observable outcome.

For create flows:

submit

- observe resulting state
- identify entity/result
- verify demonstrated values or structural witness

Only then should the workflow be promoted to certified.

6. Execution contract

Execution is responsible for producing the authoritative browser trace.

Execution should:

1. start from a fresh browser session;
2. authenticate;
3. establish the required initial state;
4. execute the workflow using the interaction engine;
5. progressively discover hidden controls when required;
6. re-observe after every meaningful state change;
7. verify each critical action;
8. capture evidence;
9. produce the authoritative DemoTrace.

Execution must never claim success based solely on:

click dispatched

It requires:

action dispatched

+

expected state transition

+

independent verification

The trace is the source of truth for downstream presentation.

7. Narration contract

Narration must never invent what execution did.

Narration should consume:

verified trace

+

approved storyboard

+

evidence

Every narration claim must be grounded in observed evidence.

If execution did not successfully perform an action:

do not narrate it as completed

The planning storyboard can provide intended editorial structure, but the final narration must bind to actual successful trace events.

Avoid rebuilding deterministic skeleton narration after execution.

8. Rendering contract

Rendering is deterministic composition.

Rendering must NOT decide:

- whether an action succeeded
- whether a workflow happened
- what the user intended
- whether a button should have been clicked

Rendering consumes:

verified trace
+
presentation plan
+
captions
+
source recording

It produces:

candidate MP4

and only promotes it to final after successful media validation.

Rendering failures should remain rendering failures.

Do not repair interaction failures in the renderer.

9. VIDEO_QA contract

VIDEO_QA is the final delivery gate.

It should aggregate:

execution QA

story QA

editorial QA

presentation QA

synchronization QA

video QA

multimodal QA

artifact completeness

But VIDEO_QA must not become a place where upstream uncertainty is hidden.

For example:

unverified interaction

→ hard failure

missing trace evidence

→ hard failure

unsupported narration claim

→ hard failure

A beautiful MP4 is not proof that the underlying interaction succeeded.

10. Failure classification

Every failure should identify the smallest responsible layer.

Use this conceptual mapping:

DISCOVERY

- URL loading
- authentication
- observation
- provider discovery
- missing product evidence

PLANNING

- impossible workflow
- missing evidence
- unsupported capability
- rehearsal failure
- certified outcome failure

EXECUTION

- target resolution
- progressive exploration
- action dispatch
- state transition
- outcome verification
- trace integrity

NARRATION

- unsupported claim

- missing scene
- caption mismatch
- editorial failure

RENDER

- source media
- composition
- encoding
- timing/media mechanics

VIDEO_QA

- cross-layer integrity
- visual quality
- synchronization
- delivery completeness

A failure should never simply become:

GENERATION_FAILED

unless the specific root cause is genuinely unavailable.

11. Retry boundaries

Retries must happen at the smallest safe owning layer.

Examples:

Discovery/auth failure
→ DISCOVERY retry

Workflow/rehearsal failure

→ PLANNING retry

Interaction/verification failure

→ EXECUTION retry

Narration/caption failure

→ NARRATION retry

Render failure

→ RENDER retry

Final media/visual review failure

→ VIDEO_QA boundary

Do not rerun the entire pipeline for a downstream failure.

Do not rerun a dispatched mutation blindly.

Do not repeat expensive Browserbase discovery when only narration failed.

12. Separate deterministic and probabilistic components

The system should clearly distinguish:

Probabilistic

- LLM objective interpretation

- workflow planning
- candidate ranking
- semantic target selection
- UI exploration
- Stagehand assistance
- editorial writing
- multimodal review

Deterministic

- action dispatch mechanics
- safety checks
- state capture
- artifact persistence
- trace schema
- verification rules
- media composition
- synchronization checks
- delivery manifest
- retry bookkeeping

The probabilistic components may propose.

The deterministic components should validate and enforce.

13. Make failures observable

Every run should provide a clear timeline:

Stage



Sub-step



Observation



Decision



Action



Result



Verification

For example:

EXECUTION

Step 4

Intent:

Open lead actions

Observation:

4 buttons

1 "More" menu

target "Export" not visible

Decision:

Explore "More" because it is relevant to the objective

Action:

click More

Result:

menu opened

Verification:

PASS

New observation:

Export menuitem visible

If something fails:

Step 5

Intent:

Export leads

Target:

menuitem "Export"

Resolution:

ambiguous – 2 candidates

Action:

NOT DISPATCHED

Failure:

TARGET_AMBIGUOUS

Recovery:

re-observe / disambiguate

Owner:

EXECUTION

This should make debugging dramatically easier.

14. Development mode

Create/maintain a development mode that stops after:

VERIFIED TRACE

The developer should be able to test:

URL + objective

without:

- narration
- rendering
- ElevenLabs
- Remotion
- VIDEO_QA

This is the primary mode for improving browser reliability.

Only once this succeeds should the complete production pipeline be tested.

15. Full-pipeline acceptance

After the interaction engine is stable, validate the full pipeline in order:

Test 1

Discovery only

Test 2

Discovery + planning

Test 3

Discovery + planning + rehearsal

Test 4

Discovery + planning + rehearsal + execution

Test 5

... + narration

Test 6

... + rendering

Test 7

... + VIDEO_QA

Do not use only end-to-end runs to diagnose failures.

The system must allow each boundary to be validated independently.

16. Complexity policy

Do not solve reliability by adding more layers.

Before adding a new abstraction ask:

Does this eliminate a real responsibility conflict?

Does it make ownership clearer?

Does it make failure diagnosis easier?

Does it reduce coupling?

If not, do not add it.

Line-count reduction is not a goal.

Clear ownership and reliable behavior are the goals.

17. Final engineering priority

Prioritize work in this order:

1. Interaction correctness
2. Progressive UI discovery
3. Outcome verification
4. Failure observability
5. Rehearsal reliability
6. Production execution reliability

7. Planning/discovery cleanup
8. Narration correctness
9. Rendering reliability
10. Final QA/repair reliability
11. Complex applications such as n8n/Excalidraw

Do not reverse this order.

The product should first be able to reliably produce:

URL

+

objective

↓

verified interaction trace

Everything else is downstream of that capability.

So, to answer your question directly

Yes, with this addendum, the brief covers the whole problem much better.

But there is one thing I would emphasize to Cursor:

Do not interpret “fix all layers” as “rewrite all layers now.”

The correct sequence is:

PRODUCTLENS 2.0

DISCOVERY



First make the starred box reliable. Then stabilize the contracts around it. Then optimize the downstream layers.

That is different from trying to make the entire pipeline pass immediately.

And I think this is particularly important for your ProductLens 2.0 codebase because your own architecture audit already identifies large stage algorithms and overlapping responsibilities in generation, discovery, planning, execution, editorial, persistence, and rendering. Pasted markdown The right response isn't another giant rewrite; it's to **make ownership and failure boundaries explicit while fixing the interaction kernel first.**

If you give Cursor the **original brief + this addendum**, it has a much more complete picture of what we have been discussing.

  Sources

Also I asked a friend and he said that we should also be having a sync engine pipeline similar to demosmith that fixes the video caption syncing issues (currently testing is done with mute mode as elevenlabs creds are not there). Also he said that instead of processing the entire video at once we should processing it in batches of each verified architecture.

Yes — **both of your friend's points are important**, and they fit into the architecture we've already discussed. I would add them, but with one important clarification: **they are downstream of the interaction/verification engine, not replacements for it.**

1. Add a dedicated Sync Engine

Right now your pipeline is roughly:

Discovery → Planning → Execution → Narration → Render → Video QA

I would evolve it to:

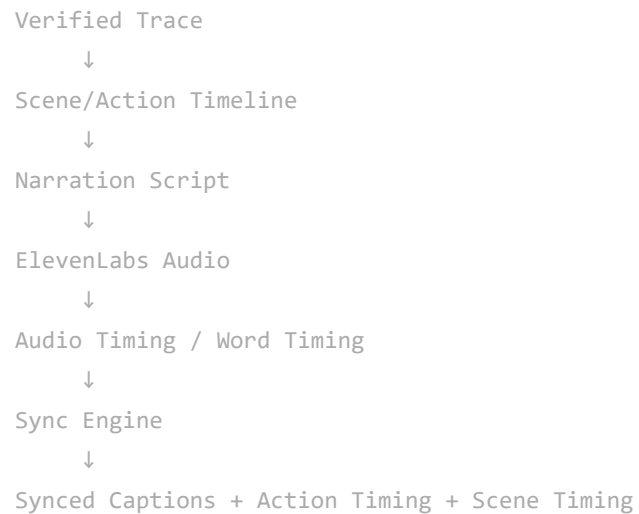
Discovery → Planning → Execution → Verified Trace → Narration → Sync Engine →
Render → Video QA

The **Sync Engine** should own the relationship between:

- verified browser actions
- timestamps
- screenshots/video segments
- narration/audio
- captions
- cursor/action highlights
- scene boundaries

The key idea is that captions should **not be independently timed based on the final video.**

Instead:



↓
Renderer

For example:

0.0s — 2.4s

Scene 1: Open Leads

2.4s — 5.8s

Scene 2: Click Add Lead

5.8s — 10.2s

Scene 3: Fill lead information

10.2s — 13.7s

Scene 4: Submit

13.7s — 15.1s

Scene 5: Verify lead created

Then narration/captions are synchronized **against those known events**, rather than trying to infer synchronization after the whole MP4 has already been rendered.

This is especially important because you eventually want the video to feel like:

narration → action → visual state change → caption

rather than:

here's a recording + here's some audio + let's try to line them up.

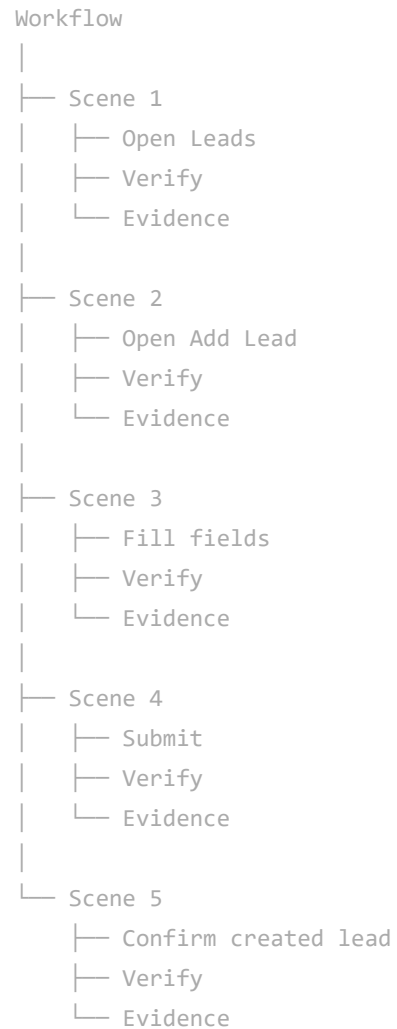
2. Your friend's batching idea is also important

I think the terminology needs to be slightly corrected.

You don't necessarily want to process arbitrary "batches of the video."

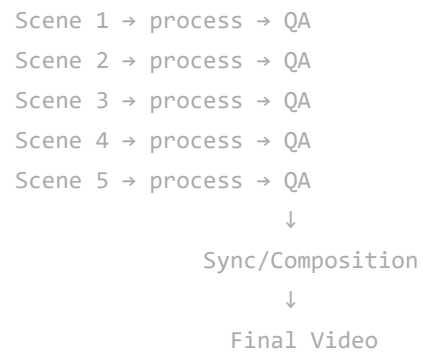
You want to process **verified workflow units / scenes independently.**

For example:



Each scene becomes a **verified production unit.**

Then:



That gives you a much better failure model.

If Scene 4 fails, you don't necessarily want:

entire 3-minute video = FAILED

You want:

Scene 1 ✓
Scene 2 ✓
Scene 3 ✓
Scene 4 ✗
Scene 5 not attempted

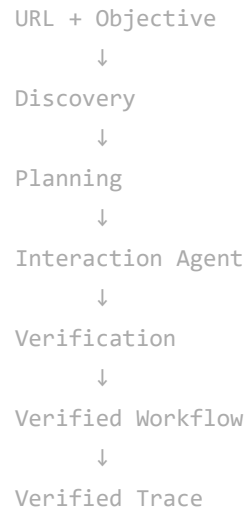
And the system can repair/re-run from the appropriate boundary.

This actually improves your architecture significantly

I would now think about ProductLens as **two pipelines**:

A. Truth pipeline

This is the most important one.



This answers:

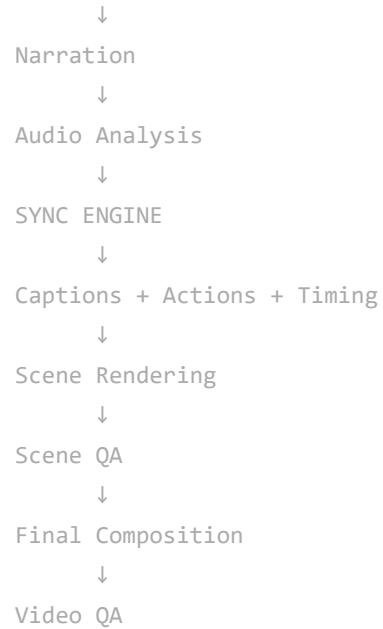
Did ProductLens actually accomplish the workflow?

Nothing downstream should be able to hide a failure here.

B. Presentation pipeline

Once you have verified truth:





This answers:

Can ProductLens turn the verified workflow into a convincing demo?

That separation is extremely valuable.

And your current ElevenLabs situation

Since you don't currently have ElevenLabs credentials, **mute-mode testing is fine for the browser/interaction pipeline**, but it means you're currently not testing the full presentation pipeline.

You should therefore have two explicit modes:

Interaction/Execution mode

Discovery
→ Planning
→ Execution
→ Verification
→ Verified Trace

Can be tested without ElevenLabs.

Full Presentation mode

Verified Trace
→ Narration
→ TTS
→ Sync
→ Captions
→ Render
→ Video QA

Requires the audio provider.

Do not let missing ElevenLabs credentials prevent you from developing the interaction engine.

But also don't consider the whole ProductLens pipeline "working" until the audio/sync path has been tested.

One more important thing

I would **not** make the Sync Engine responsible for fixing bad timing caused by upstream stages.

For example:

Execution says:

click happened at 8.2s

Actual recording:

click happened at 11.7s

The Sync Engine shouldn't just magically compensate.

That is an **execution/trace problem**.

Likewise:

Narration says:

"Now click Create"

But trace says:

Create was never successfully clicked

That's a **narration/trace-grounding problem**, not a sync problem.

The Sync Engine should receive trustworthy inputs and synchronize them.

So I'd update the implementation brief

The architecture we previously gave Cursor should now explicitly include:

DISCOVERY



PLANNING





And the **batching/scene principle** should be:

Never treat the entire generated video as one monolithic processing unit. Convert verified workflow steps into independently validated production scenes. Each scene should carry its own evidence, timing, narration, media references, QA status, and retry boundary. The final video should be a deterministic composition of successful scene artifacts.

That is a much stronger architecture than simply "generate the whole video and then run QA on the MP4."

Most importantly

I would not ask Cursor to implement all of this at once.

Your priority should remain:

1. Interaction reliability
2. Progressive UI discovery
3. Independent verification
4. Verified Trace
5. Scene/production-unit model
6. Sync Engine
7. Scene-level rendering/QA
8. Final composition

So your friend is pointing out a **real missing architectural layer**, but it doesn't change our previous conclusion: **the interaction engine remains the foundation**. Once you can reliably produce a verified trace, the sync + scene-processing architecture gives you a much more robust way to turn that truth into a high-quality demo.



Can you create a new readme covering everything we have discussed so far in detail, as we have discussed many things it would be good to have all in one doc instead of me copy pasting everything in one readme

Analysis paused ∨

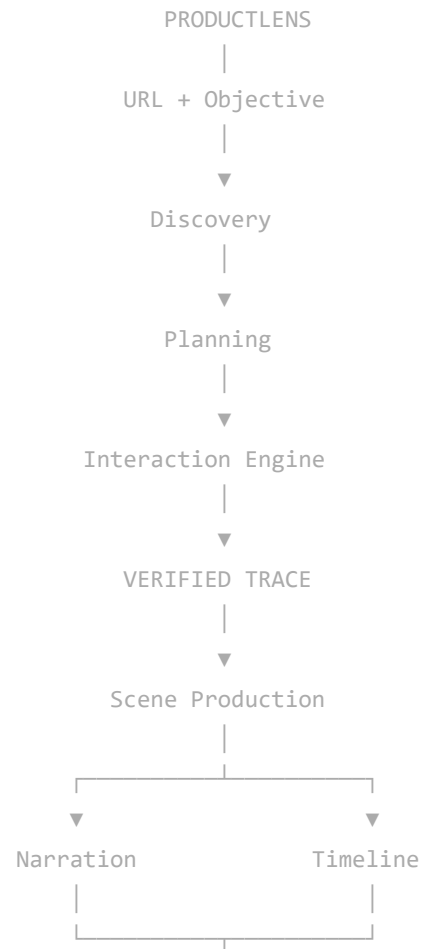
Yes. I consolidated the existing ProductLens 2.0 architecture with **everything we've discussed recently**, rather than making this just another interaction-engine prompt.

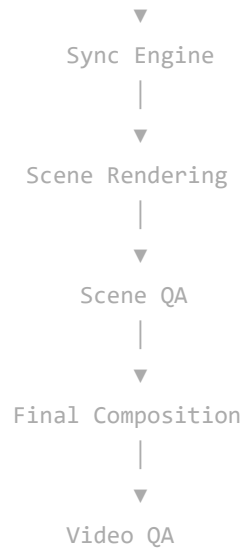
The new README covers:

- Current ProductLens architecture and durable stages
- Interaction engine redesign
- observe → resolve/explore → act → verify → re-observe
- Progressive UI discovery and hidden controls
- UI state graphs
- Exploration vs target actions
- Bounded exploration
- Target grounding and dispatch-once safety
- Independent outcome verification
- Rehearsal and certified workflows
- DemoTrace / Verified Trace as the truth boundary
- Scene-level / semantic batching
- Scene-specific retry and QA
- Video Journey Director
- Narration and caption architecture
- **Dedicated Sync Engine**
- ElevenLabs/audio timing and current caption-only mode
- Scene-level rendering
- Final video composition
- Layered QA and repair boundaries
- Persistence/artifact structure
- Browserbase / Stagehand / Playwright responsibilities

- Complex apps such as Excalidraw/n8n and canvas/graph limitations
- Architecture hotspots and refactoring strategy
- Testing and repeated reliability testing
- Implementation phases
- Explicit non-goals
- Detailed instructions for Cursor on how to implement without blindly rewriting the codebase

The most important architectural distinction in the README is now:





This is deliberately **not a giant "rewrite ProductLens" prompt**. It tells Cursor to first audit the existing implementation, preserve good pieces, identify missing/duplicated responsibilities, and then implement the changes in stages.

Download the unified README: [📄 PRODUCTLENS 2 UNIFIED README.md](#)

I would use this as the **new master README/instruction document for the ProductLens codebase**, replacing the need to keep multiple separate prompts and addenda.



ChatGPT is AI and can make mistakes.